LAB_06

Kenya Sanchez

Question 1: Write a function add() to add some numbers together. Make sure to add comment(s) to the function explaining why you do things.

Question 1a: Your first version of the function should add two input numbers together; e.g. add(4, 7) should return 11

Answer Q1a:

```
add_two <- function(x,y) {x+y}
# Adding variables x and y
```

Testing my function:

```
add_two(4,7)
```

```
[1] 11
```

```
add_two(314159,909406)
```

```
[1] 1223565
```

```
add_two(y=5, x=7)
```

```
[1] 12
```

Question 1b: Your second version of the function should add any numbers together that the user inputs; e.g. add(1, 2, 3, -4) should return 2.

Answer Q1b:

```r
add_multi <- function(x,...)
{sum(x,...)}

 #Using Sum() becuase we are adding multiple numbers

add_multi(1,2,3,-4)
```

```
[1] 2
```

Question 2: Write a function generate_dna() that generates a random DNA sequence of a length input by the user; e.g. generate_dna(11) may return "ATTTAAGGCTG". Make sure to add comments to the function. Play with functions sample() and paste() in your R-Studio Console to see how they can help here.

```r
generate_dna <- function(x) {
  # Generating a nucleotide vector
  nucs <- c("A", "T", "G", "C")
  # uysing sample to pull out random nucleotides
  nucs_vector <- sample(nucs, x, replace=TRUE)
  # Generating a dna string from nucs_vector
  paste(nucs_vector, collapse="")
}

# Testing my generate_dna() function
generate_dna(34)
```

```
[1] "ACATCGGACATGATTGCCATTCTAGTCACTTATA"
```

Question 3: Write a function generate_protein() that generates a random peptide/protein sequence of a length input by the user; e.g. generate_protein(6) may return "WQRTAG". Make sure to use the single-letter code for all 20 amino acids and use no other letters (see list of amino acids at the bottom of the page); make sure that individual amino acids can appear more than once. Make sure to add comments to the function.

```r
  generate_protein <- function(x) {
    #Generating an amino acid vector
    aa <- c("A", "R", "N", "D", "C", "E", "Q", "G", "H", "I", "L", "K", "M", "F",
            "P", "S", "T", "W", "Y", "V")
    #Using sample to pull out random peptide sequence (random amino acids)
   aa_vector <- sample(aa, x, replace=TRUE)
```

```
   #Generating a peptide/protein string form aa_vector
   paste(aa_vector, collapse="")
  }

#Testing my generat_protein() function
generate_protein(12)
```

```
[1] "DMPLCNLLWCRF"
```

Question 4: Use your generate_protein() function to generate peptides of lengths 5, 7, 9, and 11 amino acids. Take advantage of the base R function sapply() for this.

```
#Vector of the desired peptide lengths
lengths <- c(5,7,9,11)
#Generate peptide sequence for each length in vector
peptides <- sapply(lengths, generate_protein)

#Testung my generated peptide sequences
peptides
```

```
[1] "DYLLA"       "TTRCQHR"     "NSSDRLDLD"   "SMFGHMASHKR"
```

Question 5: Take each of your peptides from Q4 and run a BLASTP search (https://blast.ncbi.nlm.nih.gov/Blast.cgi) selecting the Non-redundant protein sequences (nr) database and homo sapiens (taxid: 9606) organism. For each search, before clicking BLAST check the Show results in a new window tab next to the BLAST button to easily allow you to run the blast search for all four peptides at the same time in four different tabs. Once each search is done, for the top aligned protein for each peptide, list the percent identity (Per. Ident) times coverage (Query Cover) (for example, in the example below, 100% Query Cover times 85.71% Per. Ident = 100% x 85.71% = 1.00 x 0.8571 x 100% = 85.71%); make sure to type out your calculations in your Quarto document [note: if any of your values are below 30% you are probably miscalculating something]. Upload your numbers for each peptide (length (aa) and max identity (%)) into the shared class excel data sheet (see Data Sheet link in the Canvas assignment). Also, for each datapoint, add your initials in the "Your_Initials" column of the data sheet. Before adding your initials, scan through other used initials; if anyone has the same initials as you, modify your initials to make them unique (your initials will be used below to identify your specific data points in a ggplot graph).

Peptide: KTHLM

> Top Aligned Protein: chromodomain helicase DNA binding protein 7 isoform CRA_e [Homo sapiens]

Percent Identity x Query Coverage

(100%)x(100%) = (1.00 x 1.00 x 100) = 100%

Peptide: LNFHKYG

Top Aligned Protein: immunoglobulin heavy chain junction region [Homo sapiens]

Percent Identity x Query Coverage

(85.71%)x(100%) = (0.8571 x 1.00 x 100) = 85.71%

Peptide: ESKVWDWQC

Top Aligned Protein: immunoglobulin heavy chain junction region [Homo sapiens]

Percent Identity x Query Coverage

(100%)x(56%) = (1.00 x 0.56 x 100) = 56.00%

Peptide: FESEVNYQIVM

Top Aligned Protein: sushi, von Willebrand factor type A, EGF and pentraxin domain containing 1 [Homo sapiens]

Percent Identity x Query Coverage

(87.50%)x(73%) = (0.8750 x 0.73 x 100) = 63.875%

Question 6: Once there is a good amount of data in the shared excel sheet, export the sheet as a CSV UTF-8 file (Export option under the File tab) and save the file to your class working folder (i.e. the folder in which you are currently generating a Quarto document). In R-Studio, upload the csv file into a data frame using the command peptide_df <- read.csv("your_filename"). Use the head() function to list the first 10 rows (note that the default for head() is 6) of the peptide_df dataframe as a test that everything looks good.

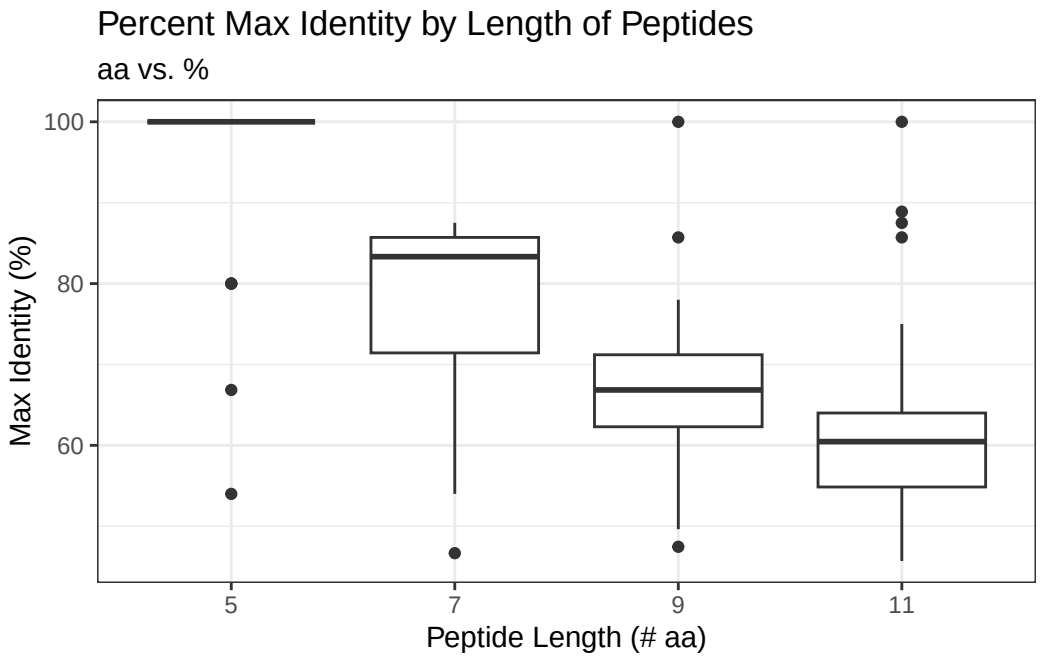
```
# Create a vector for class excel sheet for peptide Max Identity%
peptide_df <- read.csv("BIMM143_peptide.csv")
head(peptide_df,10)
```

```
   Length_aa Max_Identity_perc Your_Initials  X X.1 X.2 X.3 X.4 X.5 X.6 X.7 X.8
1          5            100.00           JLA NA  NA  NA  NA  NA  NA  NA  NA  NA
2          7             85.71           JLA NA  NA  NA  NA  NA  NA  NA  NA  NA
3          9             66.85           JLA NA  NA  NA  NA  NA  NA  NA  NA  NA
4         11             55.00           JLA NA  NA  NA  NA  NA  NA  NA  NA  NA
5          5             80.00           SKB NA  NA  NA  NA  NA  NA  NA  NA  NA
```

```
6           7              85.71            SKB NA  NA  NA  NA  NA  NA  NA  NA  NA
7           9              66.85            SKB NA  NA  NA  NA  NA  NA  NA  NA  NA
8          11              64.00            SKB NA  NA  NA  NA  NA  NA  NA  NA  NA
9           5             100.00            KET NA  NA  NA  NA  NA  NA  NA  NA  NA
10          7              85.71            KET NA  NA  NA  NA  NA  NA  NA  NA  NA
   X.9 X.10 X.11 X.12 X.13 X.14 X.15 X.16 X.17 X.18
1   NA   NA   NA   NA   NA   NA   NA   NA   NA   NA
2   NA   NA   NA   NA   NA   NA   NA   NA   NA   NA
3   NA   NA   NA   NA   NA   NA   NA   NA   NA   NA
4   NA   NA   NA   NA   NA   NA   NA   NA   NA   NA
5   NA   NA   NA   NA   NA   NA   NA   NA   NA   NA
6   NA   NA   NA   NA   NA   NA   NA   NA   NA   NA
7   NA   NA   NA   NA   NA   NA   NA   NA   NA   NA
8   NA   NA   NA   NA   NA   NA   NA   NA   NA   NA
9   NA   NA   NA   NA   NA   NA   NA   NA   NA   NA
10  NA   NA   NA   NA   NA   NA   NA   NA   NA   NA
```

Question 7: Use ggplot2 to generate a box plot with Length_aa on the x-axis and Max_Identity_perc on the y-axis. Since the box plot will require categorical values for Length_aa, you may have to specify the x-axis as as.factor(Length_aa). Feel free to pretty up the plot by adding your own x/y-axis labels, graph title, etc using the labs() function and/or using your favorite theme using a theme() function (see the data-visualization_ggplot.pdf cheat sheet in Canvas for options). Optional: you will reuse this plot a few times below, so your subsequent coding will be simpler if you assign your box plot to an object such as 'p' (p <- ggplot(peptide_df) +….etc)

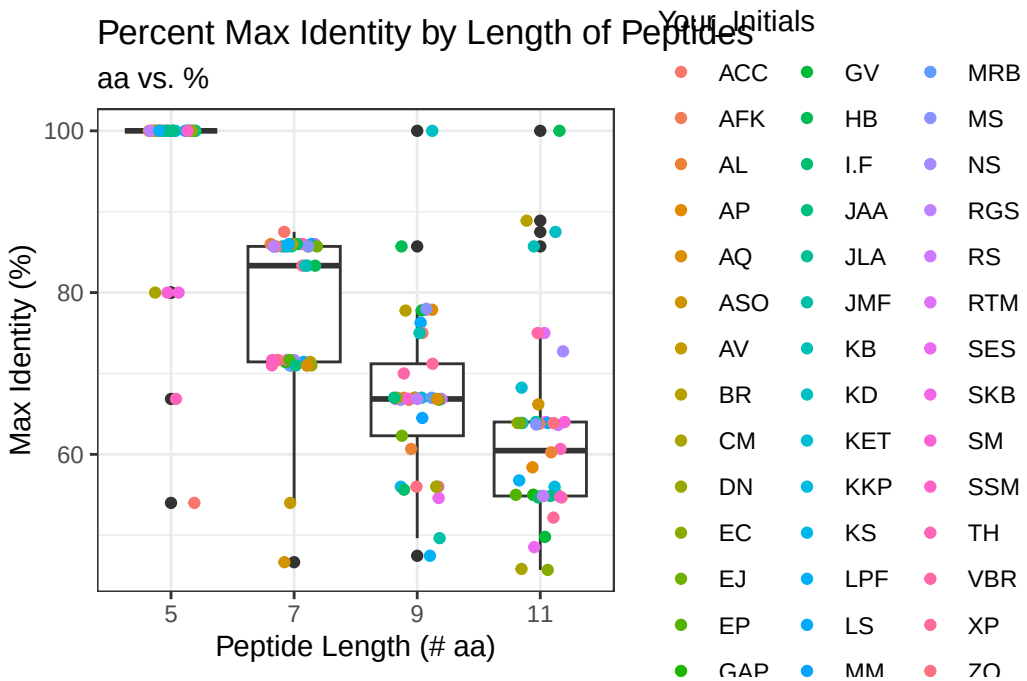
```
# Generate a box plot for each peptide length
# Assign labels and theme for better analysis and visuals
library(ggplot2)
ggplot(peptide_df) + aes(x=factor(Length_aa), y=Max_Identity_perc) +
  labs(title="Percent Max Identity by Length of Peptides", subtitle= "aa vs. %",
       x="Peptide Length (# aa)", y="Max Identity (%)") +
  geom_boxplot() +
  theme_bw()
```




```
# Make the chart a vector, more accessible for future questions
p <-ggplot(peptide_df) + aes(x=factor(Length_aa), y=Max_Identity_perc) +
  labs(title="Percent Max Identity by Length of Peptides", subtitle= "aa vs. %",
       x="Peptide Length (# aa)", y="Max Identity (%)") +
  geom_boxplot() +
  theme_bw()
```

Question 8a: Let's add a geom_jitter() layer to your box plot to visualize the individual data points on top of the box plot and highlight your own data points. First, add a geom_jitter() layer to your box plot with each data point colored according to students' initials by adding the following additional line to your plot: + geom_jitter(aes(col=Your_Initials)). Feel free to narrow the spread of the jitter points using the width argument (e.g. width = 0.2). You can find your own datapoints this way via the color legend, but it looks rather messy.

```
# Include fucntion that assigns color to every set of Initials
p + geom_jitter(aes(col=Your_Initials), width=0.2)
```



Question 8b.i: Now let's control the coloring ourselves to better highlight your own data points. We can do this by generating a vector with a color for each datapoint (we will use your favorite color for your datapoints and "GREY" for all other datapoints). You can then use this vector to assign colors with the geom_jitter() call. First, use the rep() function to generate a vector consisting of as many repeats of "GREY" as there are datapoints in the peptide_df data frame. Assign this vector to an object you name color: i.e. color <- c(rep(,)) (fill in the blanks). Once you have generated the color object, use the table() function to ensure that you have the correct number of instances of "GREY" in color. Hints for (i): Use your R-Studio Console to test the outputs of rep("GREY", 2), dim(peptide_df), and dim(peptide_df)[1], and make sure to look at documentation for the dim() function (?dim) to learn what the output numbers represent.

```
# Make all colors grey so we can then select only 1 Initial to color
color <-  c(rep("GREY", dim(peptide_df)[1]))
table(color)
```

```
color
GREY
 168
```

Question 8b.ii: Next, use the Your_Initials column in the peptide_df dataframe (which can be called as peptide_df$Your_Initials) to generate a TRUE/FALSE vector indicating which datapoints are yours (TRU
i.e. mydata <- (fill in the blank). Again, check that you have the correct number of TRUE and FALSE in my data usi

$Use your R-Studio Console to test the outputs of c("JLA","MK","BG") == "JLA" and of peptide_df$Your_Initials

```
# Make my initials the selecible marker so the box plot will only color
# my points on the graph
mydata <- peptide_df$Your_Initials=="KS"

table(mydata)
```

```
mydata
FALSE  TRUE
  164     4
```

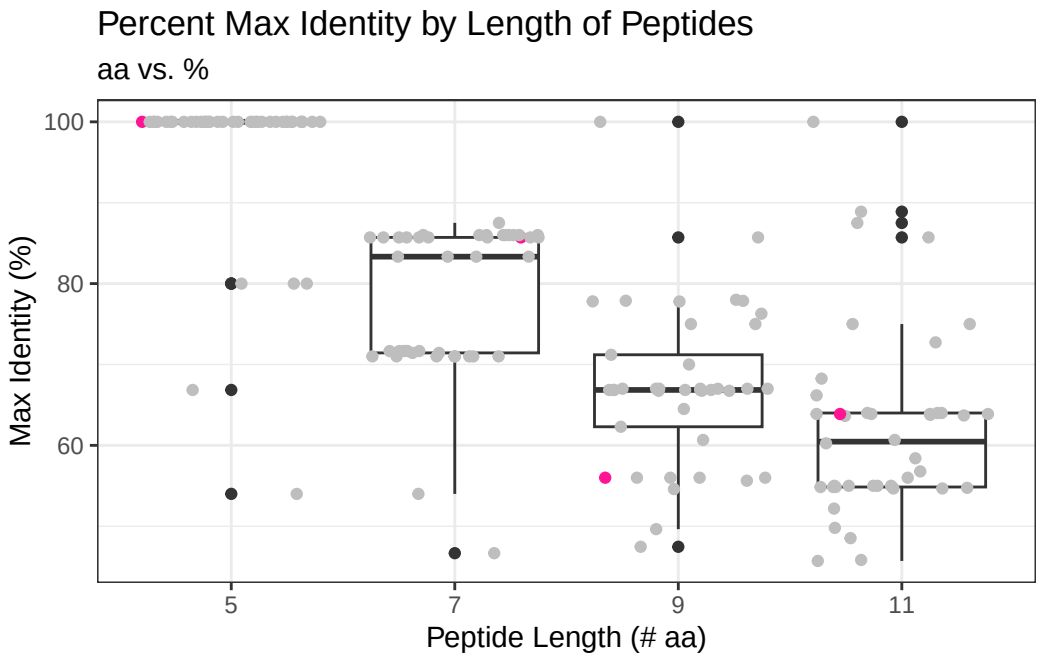
Question 8b.iii: Now, use the mydata vector to replace the "GREY" values in the color vector that correspond to your data points with your favorite color, like this: color[mydata] <- "COLOR" (replace "COLOR" with the name of your favorite color; see link at the bottom of the page to a webpage with 657 R color names to select a cool color). Note, this will replace "GREY" with your favorite color for all the positions that are TRUE in mydata, i.e. the positions of your data points. Again, check the vector with table()

```
# Assign color to my initials
color[mydata] <- "deeppink1"
table (color[mydata])
```

```
deeppink1
        4
```

Question 8b. iv: Now you are ready to remake the plot by adding the following line to your original box plot from Q7: + geom_jitter(col=color). (Note that the color call is not run as an aesthetic here because we are now directly telling ggplot what color to use for each dot). Again, feel free to use the width argument to narrow the jitter points if you wish (e.g. width = 0.2)

```
# Plot graph with color specificity for my initials
p + geom_jitter(col=color)
```



Question 8c: Based on your boxplot, does any of your own peptide data fall outside the 25 to 75 percentile of the class's data? If yes, list which one(s).

> Yes, the peptide with a 9 amino acid length falls outside of the 25 - 75 percentile of the class's data. My peptide sequence for the length 9 amino acid boxplot falls below the 25th percentile. This means that this peptide has an overall lower percent identity than the majority of the class has for this length.

Question 9: MHC class II molecules of our immune system display 9 amino acid peptides on the surface of immune cells that, when from a foreign origin, activates a T-cell immune response. Based on your findings in Q7 and 8, speculate why those peptides are 9 amino acids long rather than, say, 5 amino acids.

> If the MHC class II molecules of our immune system displayed only 5 amino acids, they would lack binding specificity. As we see in the box plot of the peptide with length of 5 amino acids, the data lies eavily clustered at high values of ~100&; this means that the peptides at this length do not provide much variation in structural identity. The box plot display for peptides of 9 amino acid length reveals lower % identity values and a broader spread. This indicated that the longer the peptide chain is, the more stable and specific it becomes. This is important especially when dealing with MHC class II molecules because we do not want to activate T-cell immune responses at random; the 9 amino acid display ensures this process remains regulated.